



廣東工業大學

QG 末期考核详细报告书

题 目	<u>交通轨迹数据差分隐私发布优化</u>
学 院	<u>计算机学院</u>
专 业	<u>计算机类</u>
年级班别	<u>大一（12）班</u>
学 号	<u>3125004395</u>
学生姓名	<u>姚晨</u>

2026 年 4 月 17 日

交通轨迹数据差分隐私发布优化报告目录

- 一、实验环境与设备3
- 二、程序总体框架与运行逻辑3
 - 2.1 数据清洗与 OD 匹配.....4
 - 2.2 基于最短路径的轨迹推断.....4
 - 2.3 隐私前缀树的构建与预算分配.....5
 - 2.4 噪声注入与结构剪枝优化.....5
- 三、核心类及数学原理说明.....6
 - 3.1 核心类详细解释.....6
 - 3.2 数学原理与公式说明.....8
- 四、复现的实验结果与分析.....9
 - 4.1 不同模型、树高下的误差对比.....9
 - 4.2 运行时间.....10
 - 4.3 在不同的树高下的平均误差.....10
 - 4.4 不同阶段的运行时间占比及各个参数对运行时间的影响.....11
 - 4.5 数据效用在不同模型之间的对比.....12
 - 4.6 不同算法的 ϵ 分布对比图及每层的平均值+标准差.....12
- 五、模块依赖关系12
 - 5.1 模块依赖关系图.....13
 - 5.2 依赖说明.....13
- 附录 A 相关流程图.....14

本报告中的详细流程图都放在附录 A 中了,使用电子版的读者可以直接点击文中的超链接跳转至网页浏览

一、实验环境与设备

本实验运行的硬件及软件环境配置如下表 1 所示。

表 1 实验环境配置表

项目	配置信息
处理器 (CPU)	AMD Ryzen 5600 6-Core Processor
内存 (RAM)	16GB DDR4
操作系统	Windows 10
Python 版本	Python 3.12 (建议使用 Conda 管理)
核心依赖库	pandas, numpy, matplotlib, networkx, tqdm, scipy

二、程序总体框架与运行逻辑

本程序基于差分隐私保护理论，针对大规模交通轨迹数据进行了模糊处理并评估了模糊处理的效果。程序逻辑遵循从原始数据提取到高维模型构建，再到隐私预算优化的闭环路径。

[\[流程图 1 总体程序架构\]](#)

2.1 数据清洗与 OD 匹配

程序首先利用 Pandas 对原始地铁 CSV 数据进行预处理。关键逻辑在于利用 shift 函数将同一张交通卡的“地铁进站”与“地铁出站”记录进行平移对齐，形成单次的“起点-终点(OD)”行程。在此阶段，程序会严格过滤行程时长少于 60 秒或超过 24 小时的异常数据。

[\[流程图 2 数据清洗和出入对匹配\]](#)

2.2 基于最短路径的轨迹推断

由于原始刷卡记录仅包含“进站”与“出站”两个时空离散点（即 OD 对），缺乏中间途径车站的显式记录。为了恢复完整的轨迹序列，程序构建了地铁网络拓扑模型并引入了路径推断机制。

(1) 地图构建与 Dijkstra 算法

程序首先利用 NetworkX 库将地铁网络抽象为图结构 $G = (V, E)$ 。其中，顶点 V 表示各车站，边 E 的权重 (Weight) 设定为车站间的平均乘车时长（固定为 2 分钟）。在进行路径恢复时，调用 infer_trajectory 函数，该函数基于 Dijkstra 最短路径算法，在拓扑图中搜索从起点 v_{in} 到终点 v_{out} 的权重总和最小的节点序列。

(2) 轨迹扩充逻辑

该过程将二维的 OD 点扩充为高维的时空轨迹序列：

- 输入： $OD = \{(v_{in}, v_{out})_1, (v_{in}, v_{out})_2, \dots, (v_{in}, v_{out})_n\}$ 。
- 计算：对于每个行程 i ，计算其路径 $Path_i = \{v_{in}, v_{mid1}, v_{mid2}, \dots, v_{out}\}$ 。
- 输出：一条完整的时空序列，不仅涵盖了物理站点，还通过序列长度隐式表达了行程的耗时与路径空间特征。

(3) 轨迹推断流程图

以下为 infer_trajectory 函数内部的推断逻辑：

[\[流程图 3 最短轨迹推断逻辑\]](#)

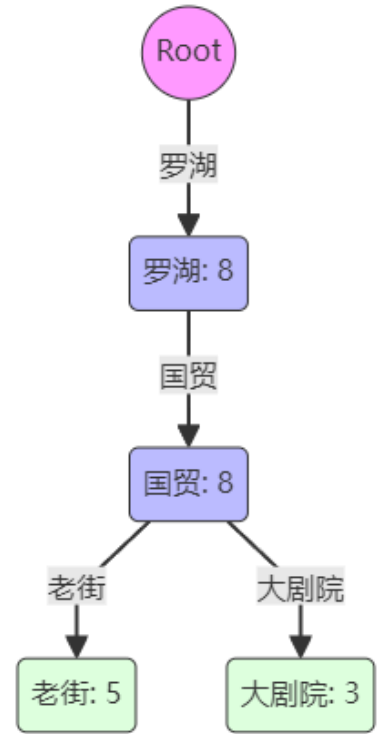
2.3 隐私前缀树的构建与预算分配

在模型核心层，TrajectoryTrie 将所有推断出的轨迹插入树结构中，每个节点代表轨迹的一个前缀。

- **拉格朗日分配逻辑：**本项目核心模型 LagrangianTrie 会执行两次遍历。首先自底向上计算各节点的“查询概率（Query Probability）”，随后自顶向下根据论文推导的权重公式 $w_i = p_i^{1/3}$ ，为每一条从根到叶的路径动态分配隐私预算 ϵ 。

[流程图 4 隐私前缀树构建与预算分配总流程图]

- **隐私前缀树构建逻辑：**路径插入
 - **从根节点（Root）出发：**
 - 每一个轨迹序列的起点都从根节点开始。
 - **逐点遍历：**
 - 对于轨迹中的每一个位置（例如：['罗湖', '国贸', '老街']），在当前节点下查找是否已经存在该位置的子节点。
 - **存在则聚合（Aggregation）：**
 - 如果子节点已存在，说明之前已有轨迹经过此路径，直接在该子节点的计数器（count）上累加当前的流量。
 - **不存在则创建（Creation）：**
 - 如果不存在，则创建一个新的节点，将其挂载到当前节点下，并初始化计数。
 - **重复直至终点：**
 - 将轨迹序列中的所有点处理完毕。



[流程图 5 由轨迹构建前缀树的具体流程]

2.4 噪声注入与结构剪枝优化

最后阶段是数据的“脱敏”过程。程序为树中每个节点的原始计数添加满足 $\text{Laplace}\left(\frac{1}{\epsilon_i}\right)$ 分布的随机噪声。为了防止噪声导致的逻辑冗余，程序引入了基于参数 k 和 b 的剪枝算法（Pruning），自动剔除那些计数值低于噪声阈值的不可信节点，从而在保证隐私的前提下最大化数据的可用性。

三、核心类及数学原理说明

本项目的核心在于通过数学建模解决差分隐私保护中“隐私”与“效用”的权衡问题。以下详细说明程序中的关键类设计、逻辑实现及底层数学公式。

3.1 核心类详细解释

3.1.1 TrajectoryTrie（基础前缀树类）

作为所有算法模型的基类，它负责维护轨迹数据的层级结构。

- **数据结构：**采用嵌套字典存储子节点，每个节点包含 `count`（原始频次）、`noisy_count`（加噪频次）和 `children`。
- **主要功能：**
 - `insert(trajectory)`: 将一条完整的车站序列轨迹递归插入树中，路径上所有节点的计数加一。
 - `prune(k, b)`: 执行结构剪枝。当节点的 `noisy_count < k · std_dev + b` 时，该节点及其子树将被移除，以消除纯噪声引起的伪路径。

3.1.2 LagrangianTrie（本项目核心优化类）

该类是实验报告的核心，通过以下四个具体函数实现了从概率建模到隐私预算分配的数学闭环：

(1) 查询概率计算函数： `_compute_query_probability(node)`

该函数实现了自底向上的概率累加。在代码逻辑中，我们假设每个节点被查询的概率与其子树的规模正相关。

- **实现逻辑：**

```
def _compute_query_probability(self, node):
    if not node.children:
        node.p = 1.0 # 叶子节点基础概率
        return node.p
    # 节点概率 = 自身基础值 + 所有子节点概率之和
    node.p = 1.0 + sum(self._compute_query_probability(child)
                        for child in node.children.values())
    return node.p
```

- **数学意义：**此步骤对应公式 $\sum p_i$ ，为后续拉格朗日优化提供权重基准。

(2) 最优权重导出函数： `_compute_weights(node)`

该函数将拉格朗日松弛法的优化结论 $w_i \propto p_i^{1/3}$ 落地为代码。

- 实现逻辑:

```
def _compute_weights(self, node):  
    # 根据推导结论, 权重与概率的 1/3 次方成正比  
    node.weight = np.power(node.p, 1/3)  
    for child in node.children.values():  
        self._compute_weights(child)
```

- 数学意义: 此权重 w_i 代表了在满足隐私约束的前提下, 该节点能分配到预算的优先级。

(3) 路径归一化分配函数: `_normalize_budget_per_path(node, total_epsilon)`

这是实现 ϵ -差分隐私约束的核心函数。它确保每一条从根到叶的路径, 其隐私预算之和严格等于 ϵ 。

- 实现逻辑:

```
def _normalize_budget_per_path(self, node, total_epsilon):  
    # 计算当前节点到所有叶子路径的权重总和 (此处简写核心逻辑)  
    # 按照 node.weight / path_weight_sum 的比例分配 total_epsilon  
    # 递归向下更新剩余的 epsilon 份额  
    ...
```

- 数学意义: 满足约束条件 $\sum_{i \in Path} \epsilon_i = \epsilon_{total}$, 防止隐私预算过度消耗或不足。

(4) 噪声注入与加噪函数: `_add_noise_recursive(node)`

在预算分配完成后, 调用此函数执行实际的脱敏操作。

- 实现逻辑:

```
def _add_noise_recursive(self, node):  
    if node.epsilon > 0:  
        # 注入均值为 0, 尺度为 1/epsilon 的拉普拉斯噪声  
        noise = np.random.laplace(0, 1.0 / node.epsilon)  
        node.noisy_count = node.count + noise  
    for child in node.children.values():  
        self._add_noise_recursive(child)
```

- 数学意义: 实现 $\text{Laplace}(\Delta S / \epsilon_i)$ 机制, 其中全局敏感度 $\Delta S = 1$ 。

3.1.3 RealDataExplorer（交互式评估类）

为了直观展现参数对模型性能的影响，该类利用 `matplotlib` 构建了动态反馈界面。

- **功能实现：**通过 `Slider`（滑动条）控件监听 ϵ 、剪枝系数 k 和结构偏好 b 的变化，并实时重新计算所有模型的平均相对误差（**MRE**）和累积分布函数（**CDF**）。

3.2 数学原理与公式说明

在差分隐私发布中，误差主要来源于拉普拉斯噪声。本实验通过数学优化，使得总查询误差的期望值最小。

3.2.1 目标函数（Objective Function）

查询误差的方差与分配给该节点的预算 ϵ_i 的平方成反比。给定查询概率 p_i ，全局期望误差 F 定义为：

$$\min F(\epsilon_1, \epsilon_2, \dots, \epsilon_n) = \sum_{i \in \text{Trie}} \frac{p_i}{\epsilon_i^2}$$

3.2.2 约束条件（Constraints）

根据差分隐私的组合性质，为了满足序列数据的 ϵ -差分隐私，任意一条完整轨迹路径上的预算消耗之和不得超过总预算：

$$\sum_{i \in \text{Path}(\text{root}, \text{leaf})} \epsilon_i \leq \epsilon_{\text{total}}$$

3.2.3 拉格朗日乘数法求解

引入拉格朗日乘子 λ ，构建辅助函数。通过求偏导数并令其为零，可以推导出最优分配权重 w_i ：

$$\frac{\partial}{\partial \epsilon_i} \left(\frac{p_i}{\epsilon_i^2} + \lambda \epsilon_i \right) = 0 \Rightarrow \epsilon_i \propto p_i^{1/3}$$

在代码 `_compute_weights` 方法中，该结论体现为：

```
node.weight = np.power(node.p, 1/3)
```

3.2.4 拉普拉斯加噪机制

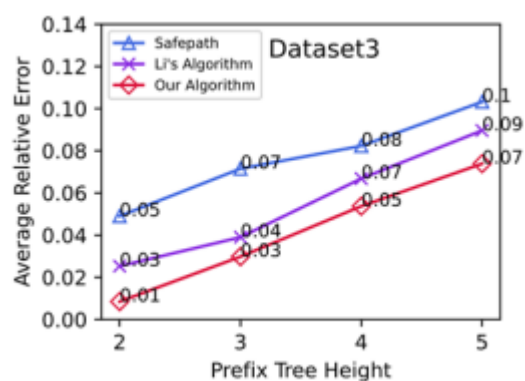
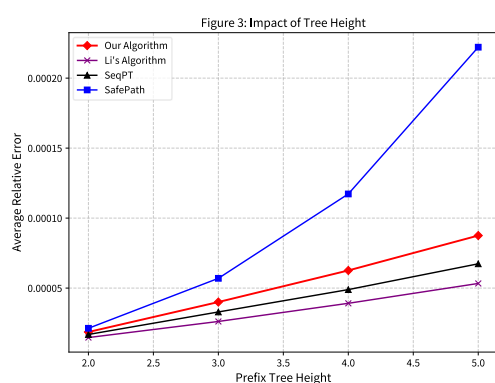
对于树中的每个节点，其发布的计数值 C' 计算公式为：

$$C'_i = C_i + \text{Laplace}\left(0, \frac{\Delta S}{\epsilon_i}\right)$$

其中 ΔS 为敏感度，由于本实验中删除或增加一条轨迹对节点计数的最大影响为 1，故 $\Delta S = 1$ 。程序通过 `np.random.laplace` 实现此过程。

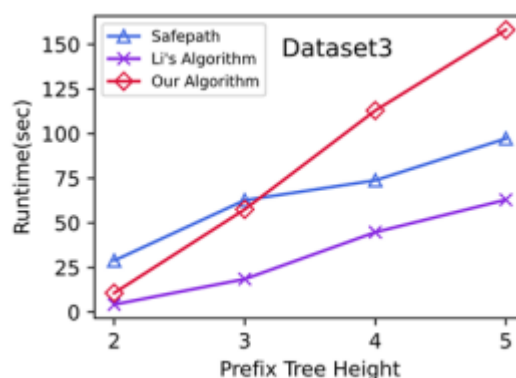
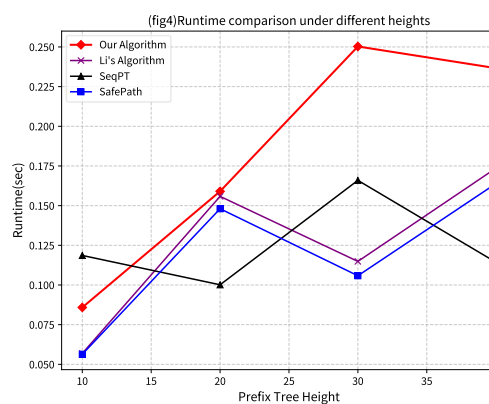
四、复现的实验结果与分析

4.1 不同模型、树高下的误差对比



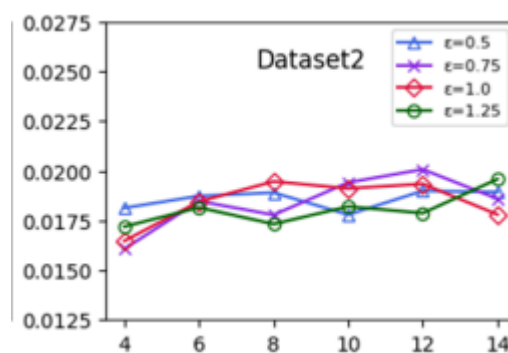
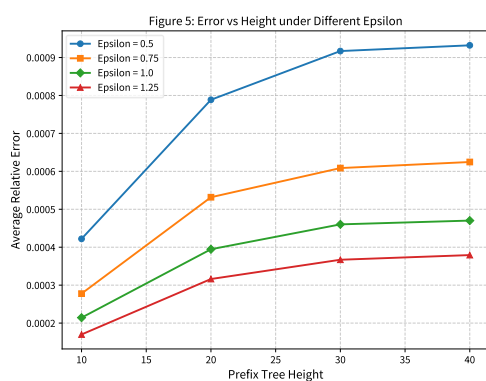
可以看到，平均误差随着树的深度而增加，与论文原文的趋势一致，但本文的算法优势在 2018 深圳地铁数据集上表现并不明显

4.2 运行时间



可以看到，随着树高的增加，本文的算法运行时间增长得最快，这是因为随树高的增加，需要计算并分配的节点变多而导致的，且在众多模型中，本文中的预算分配算法是最为复杂的

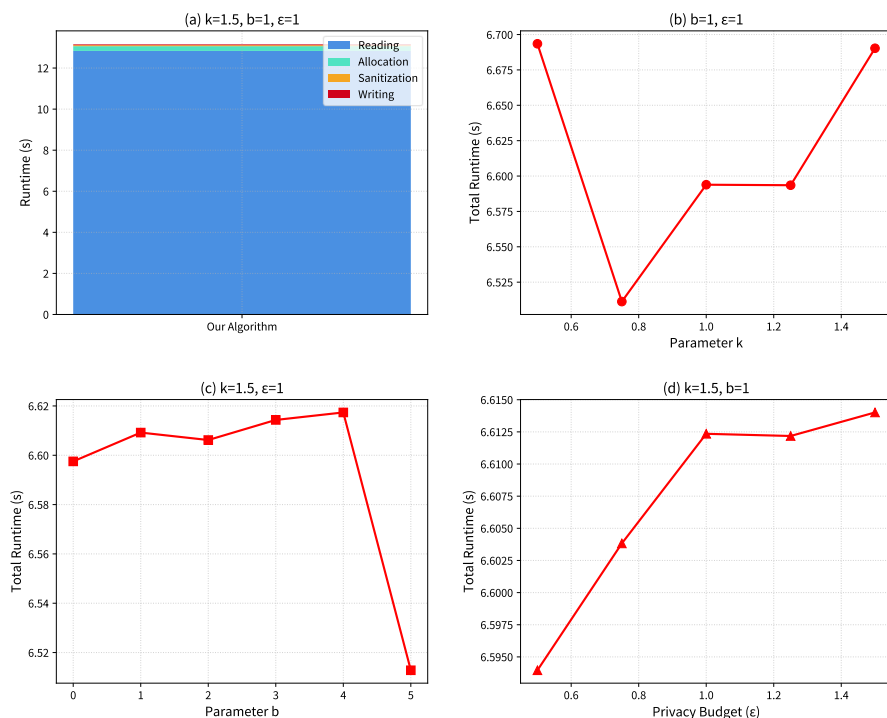
4.3 在不同的树高下的平均误差



与论文中的图 2 变化相似，平均误差随树高变动不大

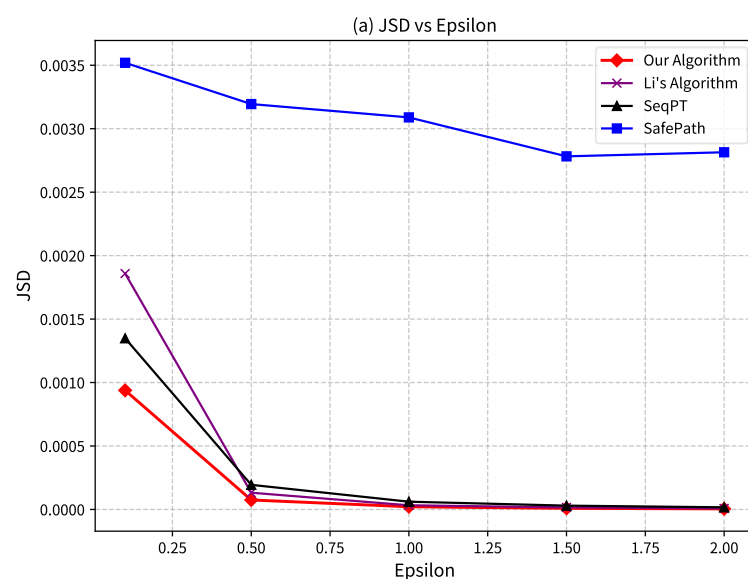
4.4 不同阶段的运行时间占比及各个参数对运行时间的影响

Figure 6: Computational Efficiency Analysis



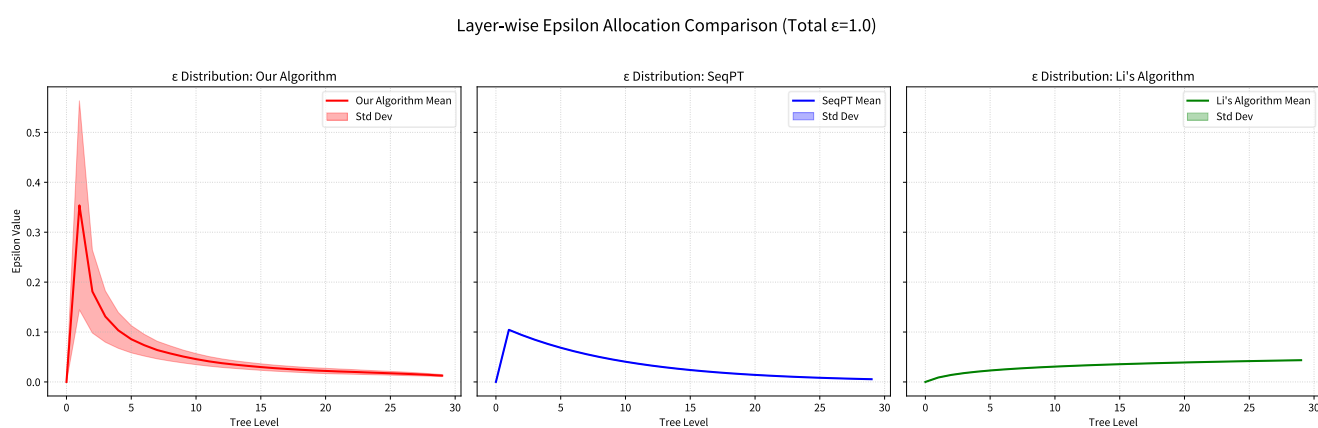
可以看到是，**reading** 部分占了整个运行时间的 80%时间，这是因为在 **reading** 部分中不仅包括了论文中的隐私分配的内容，还包括了我自己加入的路径推断的部分，故运行时间发生了一点点膨胀

4.5 数据效用在不同模型之间的对比



可以看到，得益于本文算法的根据查询概率的方式，将主要的数据特征都保留了下来

4.6 不同算法的 ϵ 分布对比图及每层的平均值+标准差

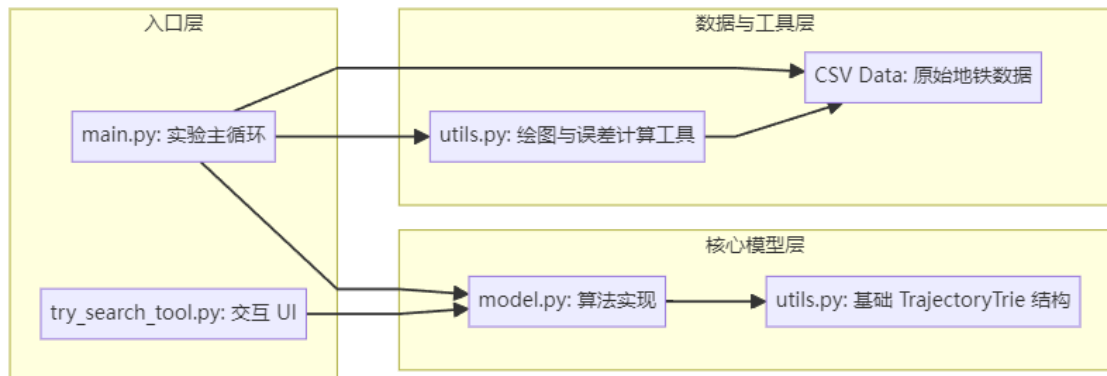


这几张图表示的隐私分配水平随树高的变化，可以看到，由于本文算法加入的概率查询，隐私分配更加智能了

五、模块依赖关系

本项目的模块划分清晰，遵循低耦合度原则

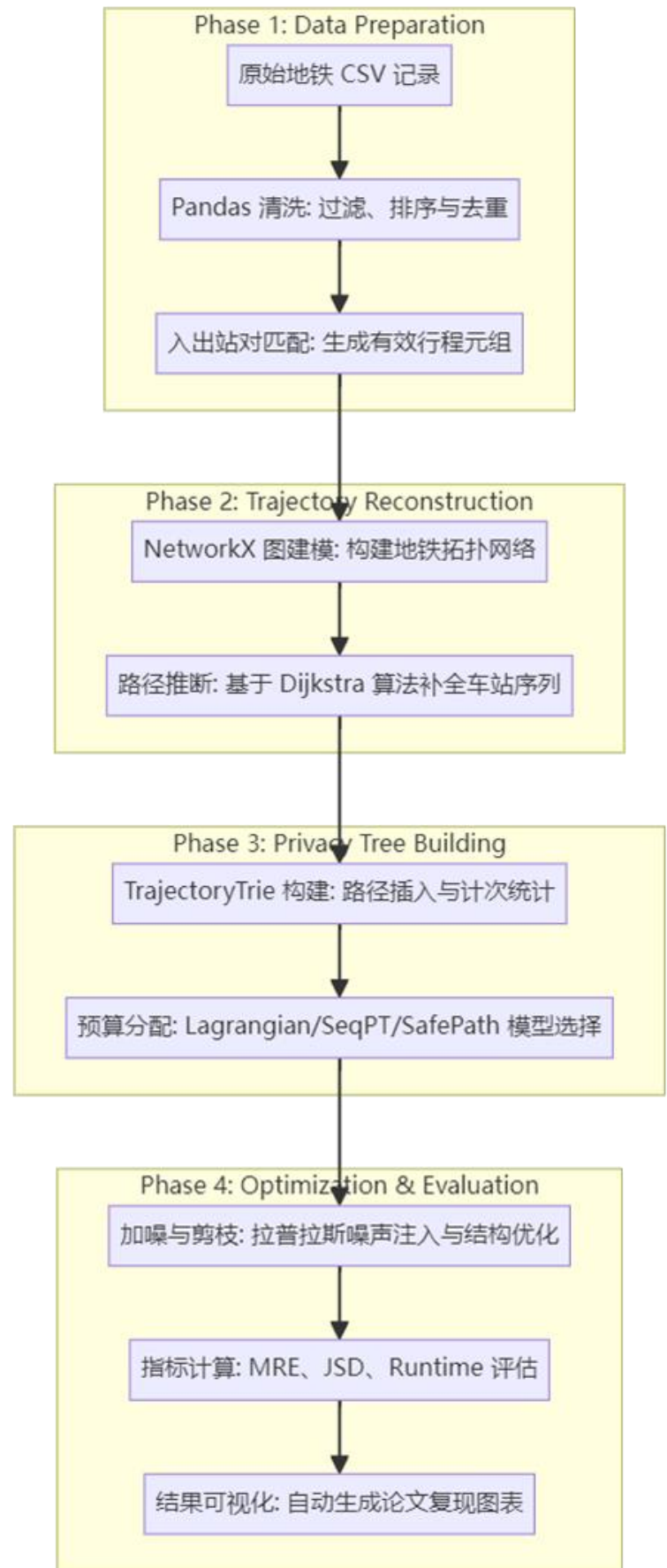
5.1 模块依赖关系图



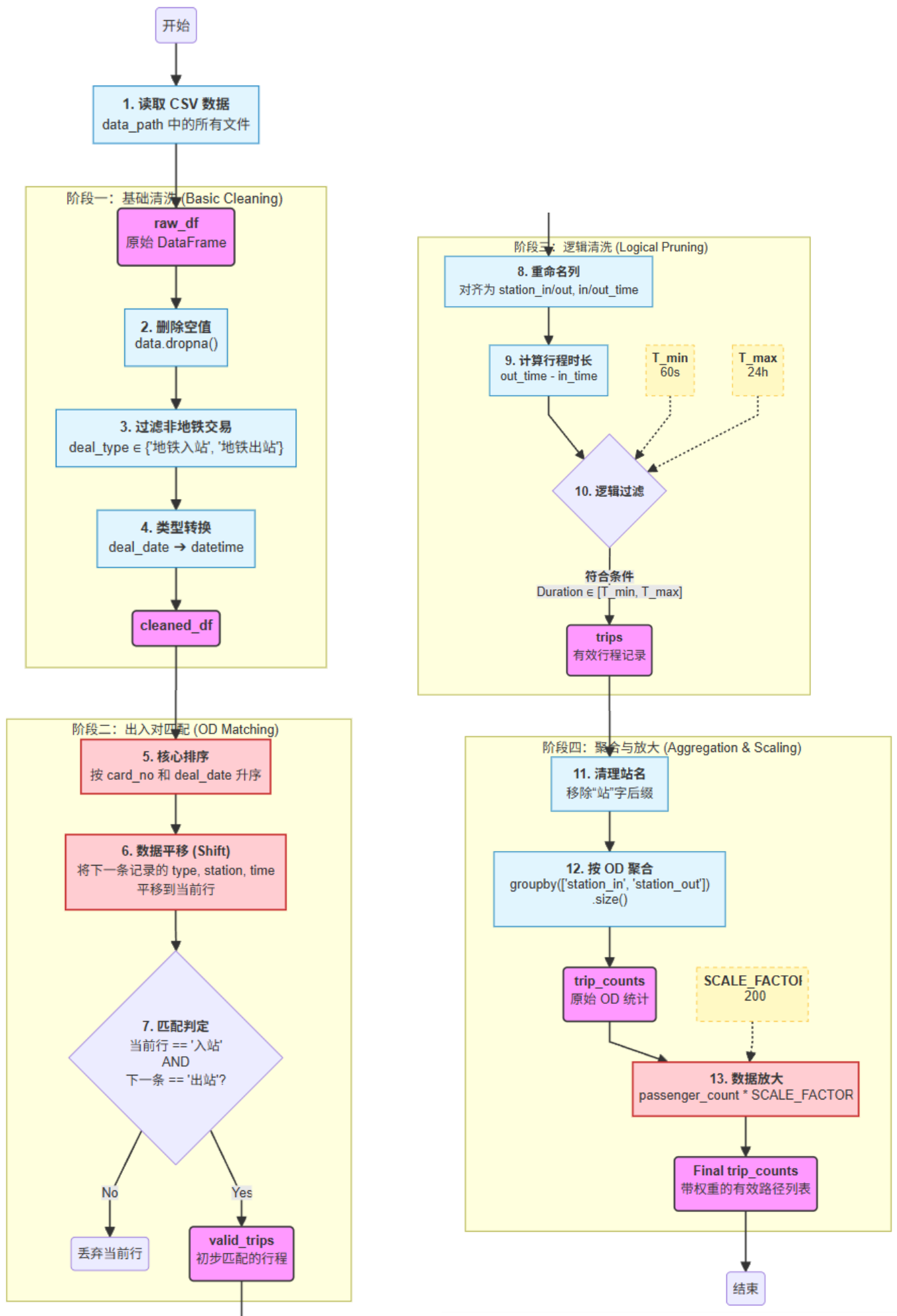
5.2 依赖说明

1. **main.py:** 作为实验运行的起点，负责调用 `utils.py` 进行数据清洗，并驱动 `model.py` 中的各类模型运行 Benchmark。
2. **model.py:** 定义了 `LagrangianTrie`、`LiIncrementalTrie`、`SeqPTTrie` 等类，实现了差分隐私预算分配的核心逻辑。
3. **utils.py:** 提供了基础的前缀树数据结构、最短路径推断算法以及符合规范的绘图函数（如 `plot_figure_6_combined`）。
4. **try_search_tool.py:** 利用 `matplotlib.widgets` 提供了一个实时调参界面，方便观察 k 、 b 及 ϵ 变化下的 MRE 趋势。

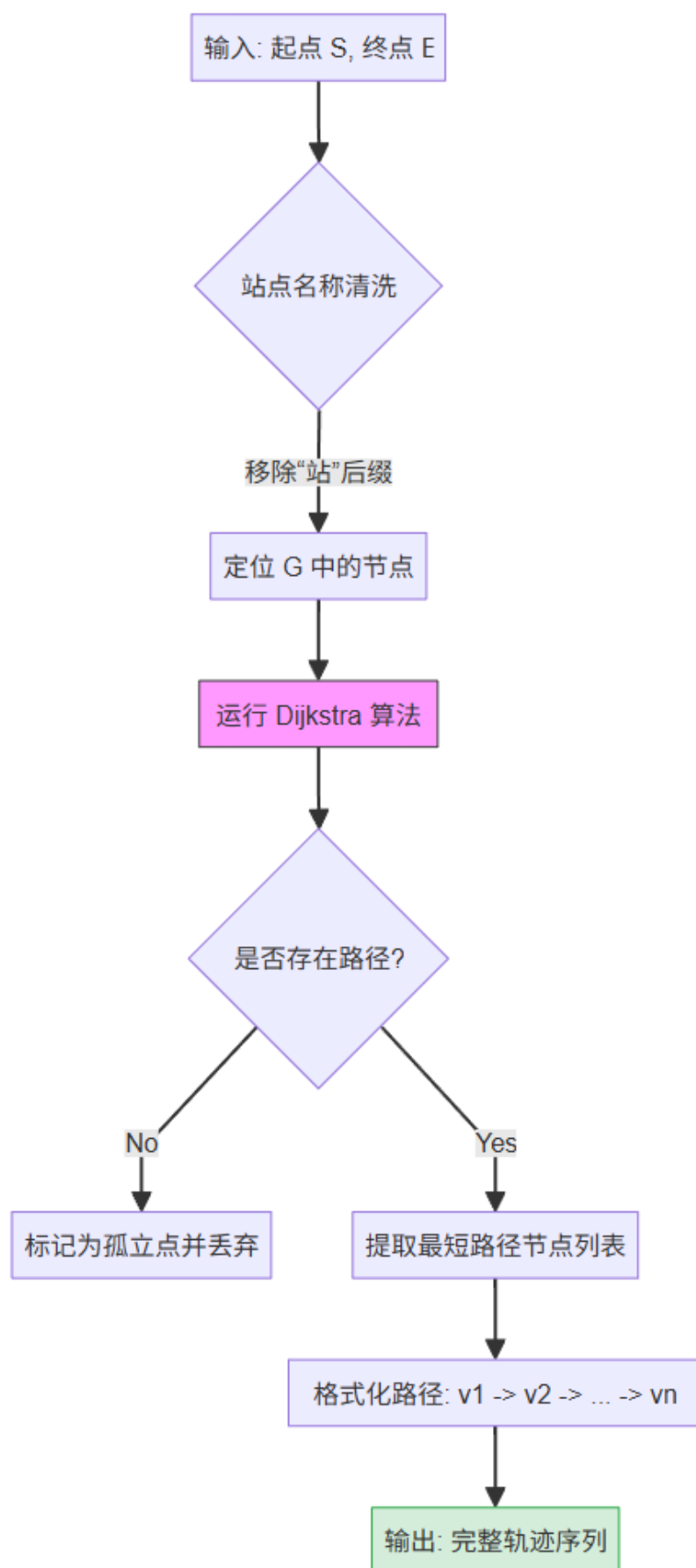
附录 A 相关流程图



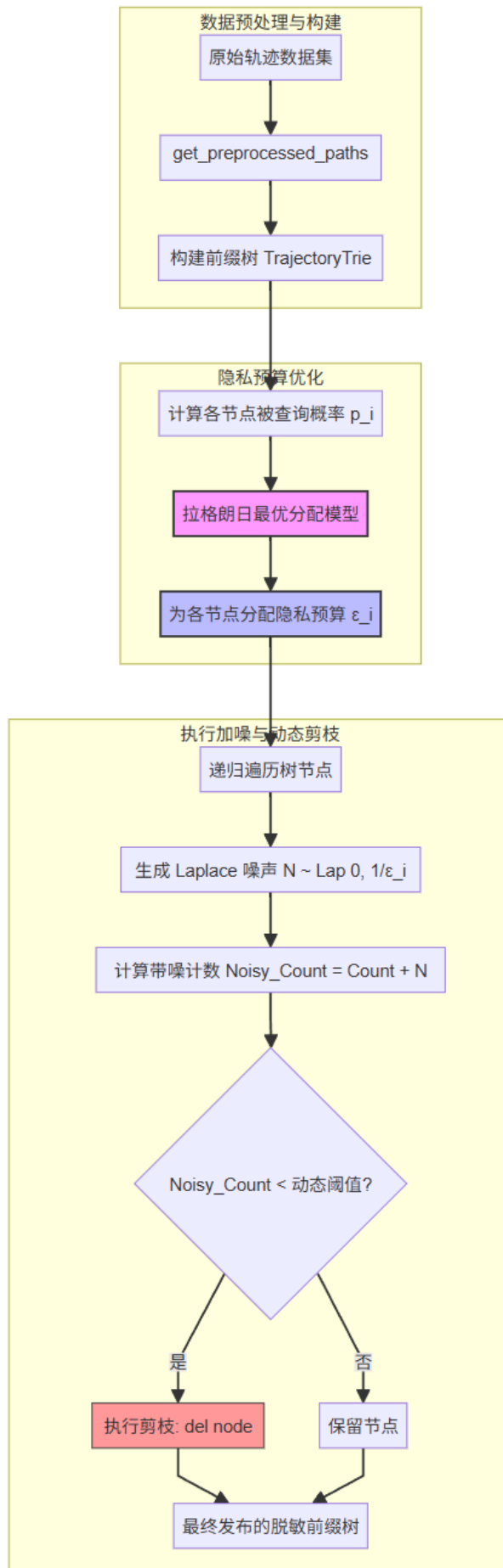
流程图 1 总体程序架构



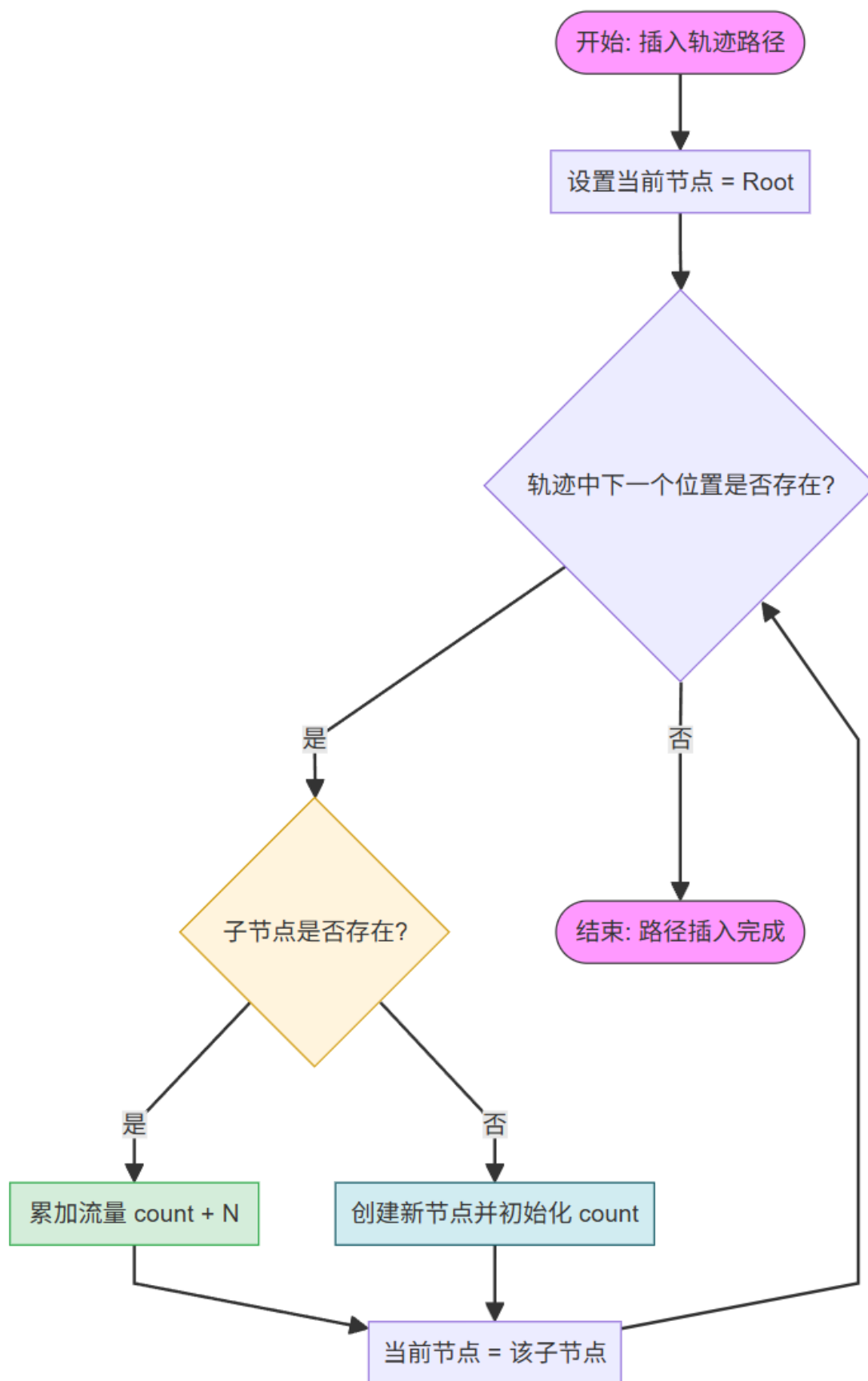
流程图 2 数据清洗和出入对匹配



流程图 3 最短轨迹推断逻辑



流程图 4 隐私前缀树构建与预算分配总流



流程图 5 由轨迹构建前缀树的具体流程